

Technical Design Document: Supervised Fine-Tuning (SFT) Module (`sft_training.py`)

1. Executive Summary & Architecture Overview

The `sft_training.py` module is a robust, production-ready training engine designed to adapt pretrained causal Large Language Models (LLMs) to follow user instructions. In the broader LLM engineering pipeline (pre-training → **SFT** → RLHF/DPO → Inference), SFT bridges the gap between raw next-token prediction and conversational alignment.

Key Engineering Objectives:

- **Precision Loss Masking:** Ensures gradients are computed strictly on assistant response tokens, completely ignoring user prompt tokens via `IGNORE_INDEX` (`= -100`).
- **Dynamic Padding & Efficiency:** Utilizes a custom collate function to pad batch sequences to the maximum length within that specific batch rather than a static global length, minimizing wasted compute.
- **Mixed-Precision & Memory Optimization:** Supports native FP16, BF16, and FP32 execution alongside activation checkpointing (`gradient_checkpointing`) and gradient accumulation.
- **Fault-Tolerant Data Ingestion:** Safely handles malformed JSONL records during loading without crashing long-running training jobs.

2. Module Specifications & Component Breakdown

2.1 Configuration Management (`SFTConfig`)

Encapsulated via a dataclass (`@dataclass`), the configuration stores all hyperparameters, paths, and hardware settings. This enables complete reproducibility from a single CLI invocation or configuration dictionary.

- **Data & Paths:** `model_name` , `train_file` , `val_file` , `output_dir` .
- **Optimization:** `max_length` , `epochs` , `batch_size` , `grad_accum_steps` , `learning_rate` , `weight_decay` , `warmup_ratio` , `max_grad_norm` .
- **Hardware & Precision:** `dtype` (`bfloat16` , `float16` , `float32`), `gradient_checkpointing` , `num_workers` , `seed` .

2.2 Data Ingestion & Fault Tolerance (`load_jsonl`)

The dataset loader reads newline-delimited JSON (`.jsonl`) files containing `prompt` and `response` keys.

- **Error Handling:** Implements `try-except` blocks around `json.loads()` and checks for required keys. Malformed lines or missing fields emit warnings via Python's `logging` module and are skipped, ensuring multi-gigabyte training corpora do not fail midway.

2.3 Tokenization & Loss Masking (`SFTDataset`)

The core mechanism transforming raw text into tensor representations:

1. **Chat Template Formatting:** Uses the tokenizer's built-in `apply_chat_template` with `add_generation_prompt=True` to format the prompt according to the model's native chat template (e.g., Llama-3 system/user tokens).

2. **Concatenation:** Combines prompt IDs and response IDs (appended with `eos_token`) into a single sequence:

$$\text{input_ids} = [\text{prompt_tokens} \parallel \text{response_tokens} \parallel \text{eos}]$$

3. **Label Construction:** Constructs the target labels tensor by assigning `IGNORE_INDEX` (`-100`) to all prompt token positions and valid token IDs to response positions:

$$\text{labels} = [\underbrace{-100, \dots, -100}_{\text{Prompt tokens}}, \underbrace{y_1, y_2, \dots, y_m, \text{eos}}_{\text{Response tokens}}]$$

This guarantees that PyTorch's cross-entropy loss function ignores prompt tokens during backward gradient propagation.

2.4 Dynamic Batching & Padding (`build_collate_fn`)

To prevent unnecessary padding compute in batches with varying sequence lengths, the `collate` function dynamically pads `input_ids` with `pad_token_id` and `labels` with `IGNORE_INDEX` to match the maximum sequence length present in the current batch.

3. Mathematical Formulation of the SFT Objective

Given a dataset $\mathcal{D}$ of prompt-response pairs (x, y) , where x represents the prompt sequence of length T_p and y represents the response sequence of length T_r , the total sequence length is $T = T_p + T_r$.

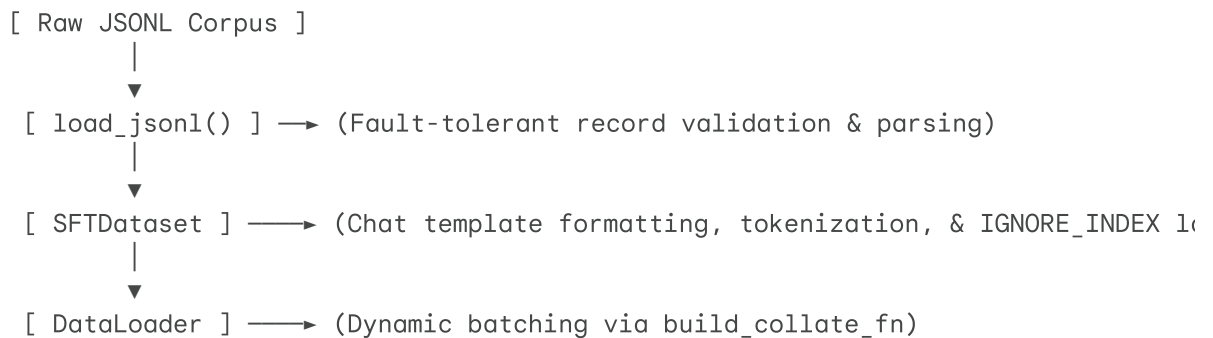
The model parameterized by weights θ computes logits over vocabulary V for each position t . The supervised fine-tuning loss $\mathcal{L}_{\text{SFT}}$ is defined as the negative log-likelihood averaged strictly over the response tokens ($m_t = 1$ for response tokens, $m_t = 0$ for prompt tokens):

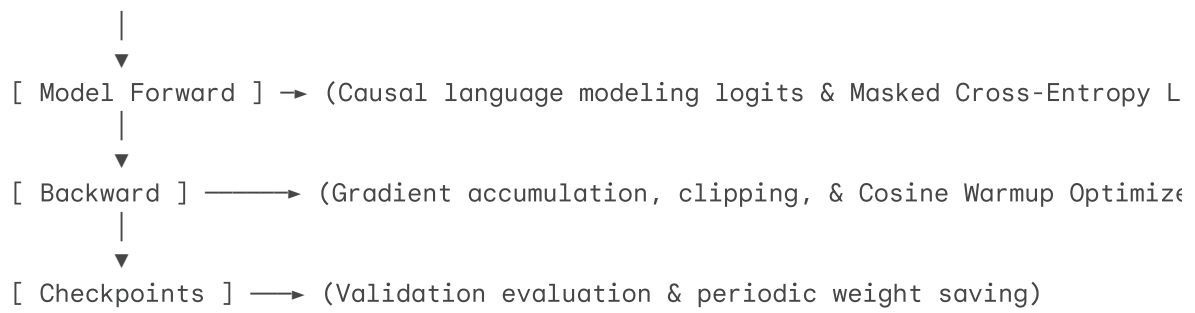
$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\sum_{t=1}^T m_t} \sum_{t=1}^T m_t \log P_{\theta}(s_t \mid s_{<t})$$

Where the token probability is given by the standard softmax over output logits z_t :

$$P_{\theta}(s_t \mid s_{<t}) = \frac{\exp(z_{t,s_t})}{\sum_{v=1}^V \exp(z_{t,v})}$$

4. Execution Workflow & Lifecycle





5. Summary and Next Steps

This design provides an optimized, reliable foundation for instruction tuning. Future extensions to this codebase can incorporate parameter-efficient fine-tuning (PEFT/LoRA) wrappers or integrate directly into Phase 2 of the RLHF pipeline (Reward Model training).

